

Ultrasonic Radar System

This project I worked on was an arduino based embedded system that is able to scan its surroundings with an ultrasonic sensor which is mounted on a sweeping servo. This gives real time visual and audio feedback through LEDS, LCD display and a buzzer. This device also streams live data to a custom PC based radar visualization that I built in a program called Processing.

How it works

This project has an ultrasonic sensor mounted on a servo motor that continuously sweeps back and forth looking to see if there are any objects nearby. The motor sweeps back forth a $\sim 170^\circ$ arc. At each angle, the Arduino sends a timed pulse and measures how long it takes to bounce back off a nearby object, converting that timing into a distance in centimeters.

Based on the measured distance the system:

- Lights green (far/no object), yellow (medium range), or red (close range) depending on distance
- Triggers a buzzer when an object is within close range
- Displays live distance and status on a 16x2 LCD screen
- Streams angle and distance data to a computer

On the computer side, a Processing sketch reads that data and renders it as a live half-circle radar display complete with a fading sweep beam, distance rings, angle markers, and detected-object dots that persist briefly before fading out.

Hardware used

- Arduino Uno
- HC-SR04 Ultrasonic Sensor
- SG90 Servo Motor
- 3x LEDs (red, yellow, green) + 220 Ω resistors
- Active buzzer
- LCD1602 display + potentiometer (for contrast)
- Breadboard + jumper wires

Software used

- Arduino IDE which I coded mostly with C and a bit of C++
- Processing (for the PC-side radar visualization)

Files

- ultrasonic_radar.ino — Arduino sketch controlling the sensor, servo, LEDs, buzzer, and LCD
- radar_display.pde — Processing sketch that reads serial data and draws the live radar display

Setup

1. Wire the components as described above (see comments in code for pin assignments)
2. Upload ultrasonic_radar.ino to the Arduino via the Arduino IDE
3. Close the Arduino Serial Monitor (only one program can use the serial port at a time)
4. Open radar_display.pde in Processing and update the port name ("COM4") to match your system
5. Run the Processing sketch to see the live radar display

What I learned

- Reading sensor data using precise pulse timing in C
- Controlling a servo motor and building a bouncing sweep algorithm
- Multi-branch conditional logic driving multiple outputs (LEDs, buzzer, LCD) simultaneously
- Interfacing with hardware libraries (LiquidCrystal, Servo)
- Streaming structured data over serial between two independent programs
- Debugging both coding logic errors and real hardware issues (wiring shorts, component polarity, serial port conflicts) independently

Future improvements

- Add a second sensor or wider sweep range for full 180° coverage
- Log detection data to a file for later analysis